MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Facultatea de Automatică şi Calculatoare

Suspensie Activă

Documentaţie de proiect — Identificarea Sistemelor

Nume: POP Raul
Specializare: Automatică şi Informatică Aplicată
Grupa: 30135

Introducere

Scopul lucrării este identificarea unui sistem de ordinul al II-lea cu poli complex conjugați, pornind de la date experimentale furnizate sub forma unor vectori de intrare și ieșire. Datele au fost achiziționate printr-un model Simulink, sistemul având un semnal de intrare de tip treaptă.

Din reprezentarea grafică a răspunsului se observă comportamentul specific unui sistem de ordinul doi, care pornește cu pantă nulă în momentul comutării semnalului de intrare.

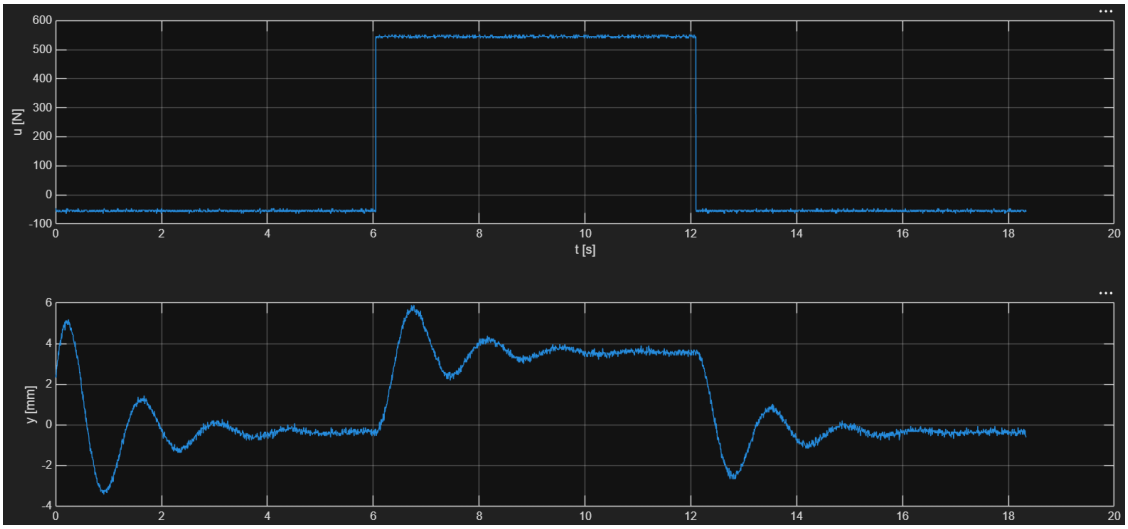


Figura 1 - Semnalele achiziționate: intrarea u(t) și ieșirea y(t).

O structură adecvată pentru acest tip de răspuns este funcția de transfer de ordinul doi, cu poli complex conjugați:

$$\mathrm{H}(s) = \frac{K{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2}$$

Problema identificării se reduce, așadar, la determinarea celor trei parametri: constanta de proporționalitate K, pulsația naturală ωn și factorul de amortizare ζ. Aceștia sunt determinați în secțiunea următoare, prin metoda regresiei liniare.

1. Metoda Regresiei Liniare

1.1 Fundamentare teoretică

Metoda regresiei liniare constă în construirea unei funcții de cost minim, cu ajutorul căreia se aproximează cât mai bine un set de date printr-o dreaptă, astfel încât eroarea rezultată să fie minimă. Funcția de cost este suma erorilor ridicate la pătrat, pentru a elimina influența semnului erorii asupra rezultatului:

$$J = \Sigma\,(y - \widehat{y})^2 = \Sigma(y - \alpha t - \beta)^2$$

unde coeficienții ce trebuie determinați sunt α și β. Pentru un cost minim, este suficient ca derivatele parțiale ale funcției J în raport cu fiecare parametru să fie egale cu zero:

$$\frac{\partial J}{\partial \alpha} = \alpha \cdot \Sigma t^2 + \beta \cdot \Sigma t - \Sigma(t \cdot y) = 0$$

$$\frac{\partial J}{\partial \beta} = \alpha \cdot \Sigma t + \beta \cdot \Sigma 1 - \Sigma y = 0$$

În urma rearanjării, sistemul de ecuații se poate scrie sub formă matriceală:

$$\begin{bmatrix} \Sigma t^2 & \Sigma t \\ \Sigma t & N \end{bmatrix} \cdot \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \Sigma (t \cdot y) \\ \Sigma y \end{bmatrix}$$

sau, echivalent:

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \Sigma t^2 & \Sigma t \\ \Sigma t & N \end{bmatrix}^{-1} \cdot \begin{bmatrix} \Sigma (t \cdot y) \\ \Sigma y \end{bmatrix}$$

1.2 Determinarea constantei de proporționalitate K

Parametrul K, numit și constanta de proporționalitate se determină ca raport între variația ieșirii și variația intrării, calculate în regim staționar:

$$K = \frac{(y_{\text{st}} - y_0)}{(u_{\text{st}} - u_0)}$$

Valorile u0, y0, ust și yst au fost calculate ca medii ale semnalelor pe intervale alese din regimul staționar, înainte și după comutarea treptei:

```
i1 = 2461;  i2 = 2907;  i3 = 5482;  i4 = 5979;

u0  = mean(u(i1:i2));   y0  = mean(y(i1:i2));
ust = mean(u(i3:i4));   yst = mean(y(i3:i4));

K = (yst - y0) / (ust - u0);
```

1.3 Determinarea parametrilor ζ și ωn

Pentru determinarea celorlalți doi parametri este necesară o succesiune de pași. Pornind de la răspunsul indicial al sistemului de ordinul doi:

$$y(t) = \mathrm{L}^{-1}\left(\mathrm{H}(s) \cdot 1/s\right)(t) = y_0 + (y_{\text{st}} - y_0)\left[1 - \frac{\mathrm{e}^{-\zeta\omega_n(t-t_0)}}{\sqrt{1-\zeta^2}} \cdot \sin\left(\omega_n\sqrt{1-\zeta^2}\,(t-t_0) + \varphi\right)\right]$$

$$|y(t) - y_{st}| = \frac{y_{st} - y_0}{\sqrt{1-\zeta^2}} e^{\mathrm{Re}\{s_{1,2}\}(t-t_0)} \left|\sin\left(Im\left\{s_{1,2}\right\}\right)(t-t_0) + \varnothing\right|$$

prin rescriere și logaritmare, se obține o relație liniară:

$$\ln\left(y(t) - y_{\text{st}}\right) = \alpha \cdot t + C, \quad unde \quad \alpha = \mathrm{Re}\left\{s_{1,2}\right\} = -\zeta\omega_n$$

Ecuația este, de fapt, ecuația unei drepte, y = a·t + b. Pentru a vizualiza mărimea y = |y(t) − yst|, se aleg valorile din palierul tranzitoriu, suficient de apropiate de stabilizare — dar nu mai mult de 1–2%, altfel logaritmarea ar introduce valori care denaturează dreapta de regresie. Prin scăderea palierului tranzitoriu cu valoarea de regim staționar, luată în modul, rămân strict „salturile", adică maximele și minimele oscilației amortizate.

```
tmin = 3033;  tmax = 6072;
y_aux = abs(y(tmin:tmax) - yst);
t_aux = t(tmin:tmax);
```

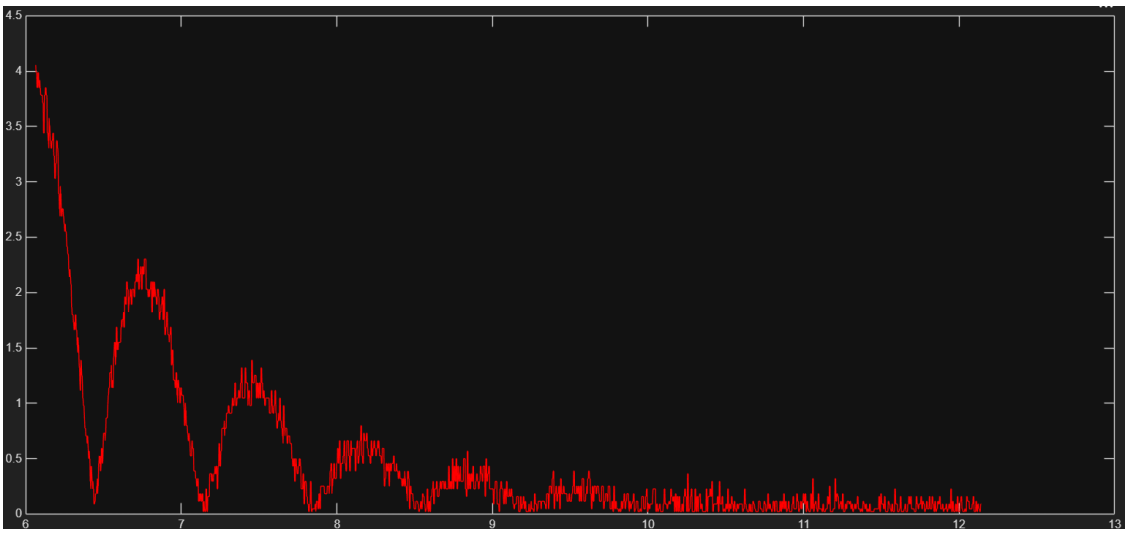


Figura 2 - Secțiunea oscilațiilor amortizate, |y(t) − yst|.

În practică sunt necesare minimum 2 puncte; pentru acest proiect au fost extrase 5, iar pe baza acestora, după logaritmare, a fost trasată dreapta de regresie cea mai fidelă setului de date.

```
im1 = 6;  im2 = 343;  im3 = 710;  im4 = 1070;  im5 = 1402;

t_reg = t_aux([im1 im2 im3 im4 im5]);
y_reg = log(y_aux([im1 im2 im3 im4 im5]));
```

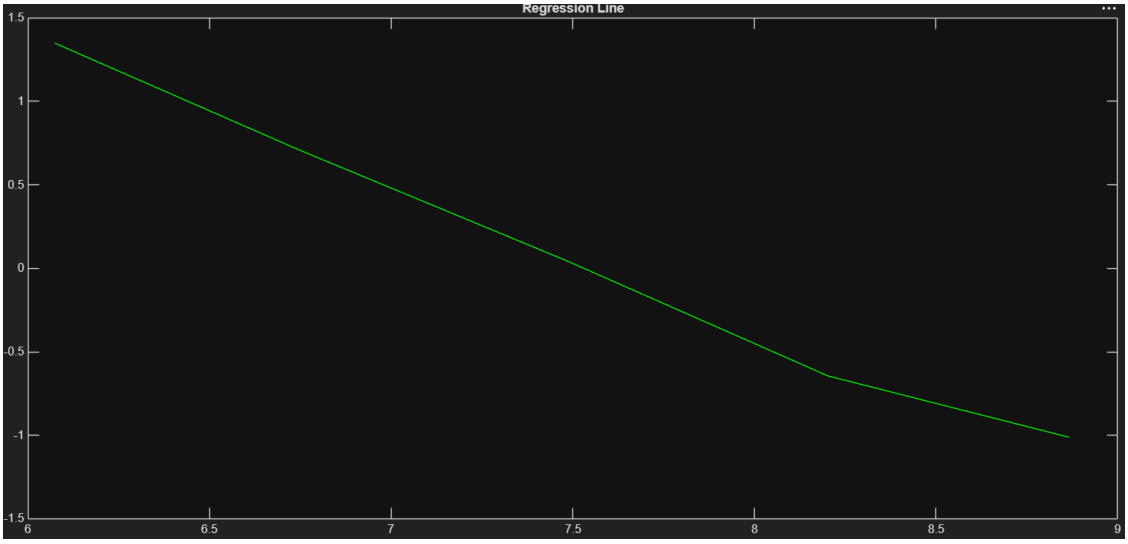


Figura 3 — Dreapta de regresie.

Modificarea pantei observată spre finalul graficului este cauzată strict de zgomotul semnalului: apropierea de valoarea staționară conduce la logaritmarea unor valori foarte mici, puternic afectate de zgomot.

Scopul este determinarea lui ωn și ζ, pe baza relațiilor:

$$\zeta = \frac{-\mathrm{Re}\{s_{1,2}\}}{\sqrt{\mathrm{Re}\{s_{1,2}\}^2 + \mathrm{Im}\{s_{1,2}\}^2}}$$

$$\omega_n = \frac{-\mathrm{Re}\{s_{1,2}\}}{\zeta}, \quad pentru \;\; \zeta \in (-1,\ 1), \;\; \zeta \neq 0$$

Așa cum s-a menționat, partea reală este α, determinată în urma regresiei. Partea imaginară se poate obține direct din pulsația de oscilație:

$$\mathrm{Im}\{s_{1,2}\} = \omega_{\mathrm{osc}} = \frac{2\pi}{\mathrm{T}_{\mathrm{osc}}}$$

iar Tosc se determină direct ca diferență între două maxime succesive ale oscilației amortizate.

1.4 Reconstrucţia modelului

Odată determinați toți parametrii (K, ζ, ωn), modelul poate fi reconstruit sub cele două forme standard: funcție de transfer şi spațiul stărilor.

Funcţia de transfer

```
H = tf(K*wn^2, [1, 2*zeta*wn, wn^2]);
y_sim_tf = lsim(H, u, t);
```

În urma simulării se obține semnalul estimat, folosind funcția lsim. Această simulare pornește din condiții inițiale nule; va fi nevoie de un anumit timp până răspunsul simulat se apropie de cel real, motiv pentru care erorile J şi eps_MPN vor fi considerabil mai mari decât în cazul simulării în spațiul stărilor (secțiunea 1.5).

Spaţiul stărilor

$$\mathrm{A} = \begin{bmatrix} 0 & 1 \\ -{\omega_n}^2 & -2\zeta\omega_n \end{bmatrix};$$

$$\mathrm{B} = \begin{bmatrix} 0 \\ K{\omega_n}^2 \end{bmatrix};$$

$$\mathrm{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}; \mathrm{D} = 0;$$

```
sys = ss(A, B, C, D);
y_sim_ss = lsim(sys, u, t, [y(1) 24.49]);
```

Pentru condițiile inițiale, poziția de start este cunoscută direct din semnalul măsurat. Pentru determinarea vitezei inițiale există două abordări: una analitică (derivarea semnalului şi extragerea primei valori) şi una numerică — metoda înjumătățirii intervalului (bisecție), folosită în acest proiect. Prin calibrare succesivă s-a ajuns la valoarea 24.49, punct dincolo de care orice altă modificare înrăutățea eroarea de validare.

1.5 Validarea modelului

Pentru verificarea modelelor propuse au fost calculaţi cei doi indicatori de performanță prezentați la cursul de Identificarea Sistemelor:

$$J = \frac{\| \varepsilon \|_2}{\sqrt{N}}$$

$$\varepsilon_{\mathrm{MPN}} = \frac{\| \varepsilon \|_2}{\| y - \bar{y} \|_2}$$

Pentru reducerea suplimentară a erorii, semnalul de ieşire măsurat a fost filtrat atât cu un filtru Butterworth de ordinul 4, cât şi cu un filtru de tip moving average.

Modelul în spaţiul stărilor

Modelul în spațiul stărilor urmărește foarte bine semnalul măsurat, atât brut, cât şi filtrat. După filtrarea Butterworth, eroarea a ajuns la eps_MPN = 1,25%, sub pragul de 10% impus.

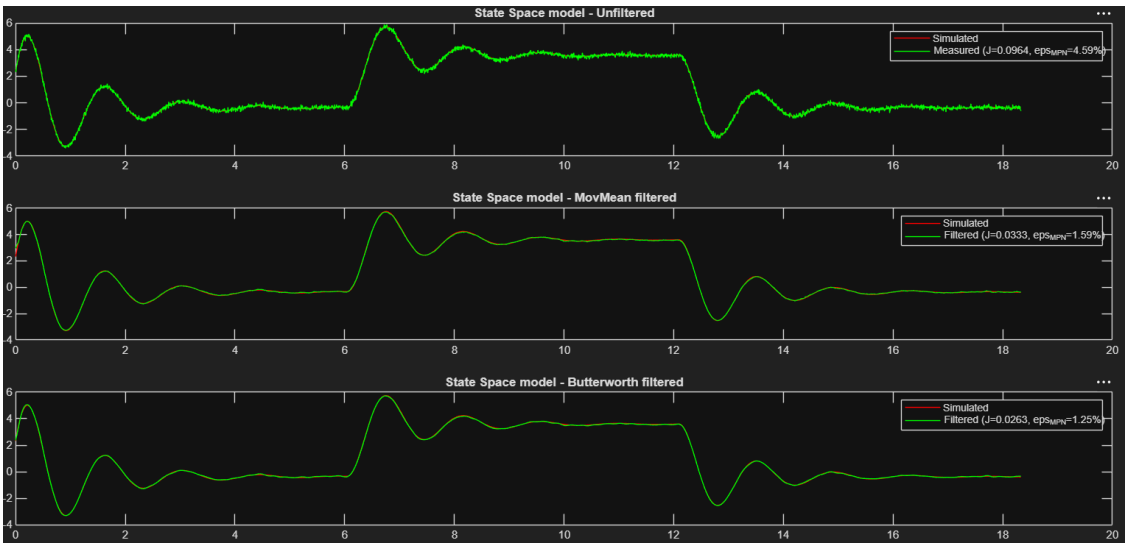


Figura 4 — Validarea modelului în spațiul stărilor.

Modelul funcție de transfer

Modelul funcție de transfer prezintă o eroare semnificativ mai mare (eps_MPN ≈ 40%), deși parametrii K, ζ și ωn sunt identici cu cei folosiți în modelul spațiul stărilor. Cauza este lipsa condițiilor inițiale la apelul funcției lsim pentru funcția de transfer — spre deosebire de simularea în spațiul stărilor, unde condiția inițială a fost specificată explicit ([y(1) 24.49]), simularea prin funcția de transfer pornește implicit din condiții inițiale nule. Eroarea mare nu persistă și pe intervale de testare neutilizate anterior la calibrare, ceea ce confirmă că nu este un defect al porțiunii de date alese, ci efectul direct al condiției inițiale nule.

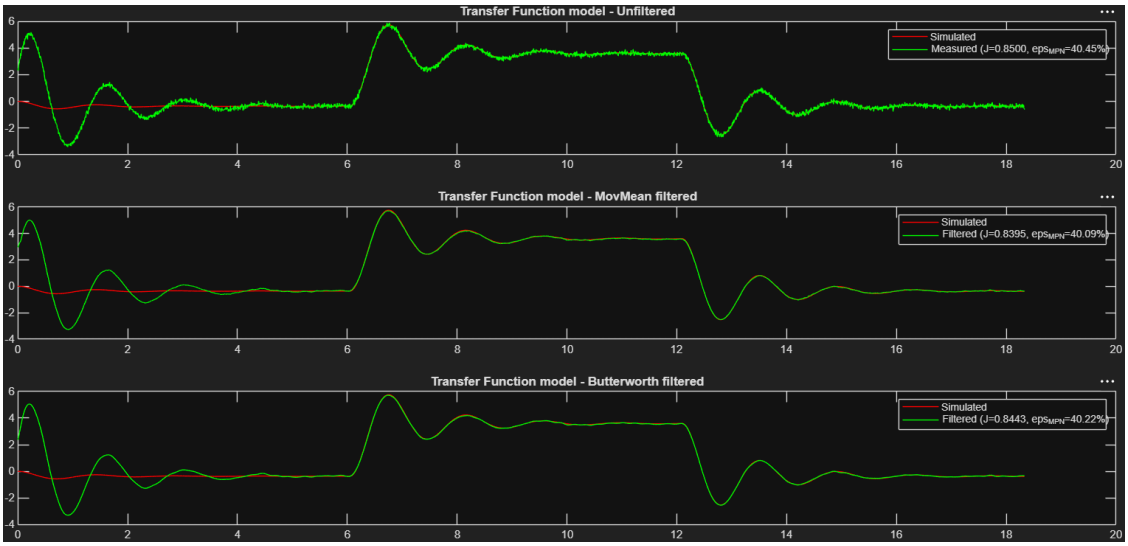


Figura 5 — Validarea modelului funcție de transfer.

Inițial, semnalul simulat oscila mai rapid decât semnalul de ieșire, fapt ce sugera ca ωn era mai mare decât ar fi trebuit. Reducerea lui a fost realizată prin creșterea perioadei de oscilatie T_{osc}, prin modificarea fină a indicilor aleși la punctele de extrem.

Anexă — Cod sursă MATLAB

Codul următor conține implementarea completă a metodei de identificare prezentate: calculul câștigului static, regresia pe palierul oscilațiilor, reconstrucția modelului sub formă de funcție de transfer și spațiul stărilor, filtrarea semnalului și calculul indicatorilor de performanță.

```
%% System identification

% Regresie formula 1 slide 21

i1 = 2461;
i2 = 2907;
i3 = 5482;
i4 = 5979;

u0  = mean(u(i1:i2));
y0  = mean(y(i1:i2));
ust = mean(u(i3:i4));
yst = mean(y(i3:i4));

K = (yst - y0)/ (ust - u0);

%%
tmin = 3033;
tmax = 6072;

y_aux = abs(y(tmin:tmax) - yst);
t_aux = t(tmin:tmax);
figure;
plot(t_aux, y_aux, 'r');

im1 = 6;
im2 = 343;
im3 = 710;
im4 = 1070;
im5 = 1402;

t_reg  = t_aux([im1 im2 im3 im4 im5]);
y_reg = log(y_aux([im1 im2 im3 im4 im5])) ;

figure;
plot(t_reg, y_reg, 'g');
title('Regression Line');

A_reg = [sum(t_reg.^2) sum(t_reg);
   sum(t_reg) length(t_reg)];
B_reg = [sum(t_reg.*y_reg);
   sum(y_reg)];
theta = A_reg\B_reg;
alpha = theta(1); %Re{s1,2}

Tosc = t(im3) - t(im1);
wosc = 2*pi/Tosc; % Im{s1,2}

zeta = - alpha/(sqrt(alpha.^2 + wosc.^2));
wn   = - alpha/zeta;

%% State Space
A = [ 0 1;
   -wn.^2 -2*zeta*wn];
B = [0;
   K*wn.^2];
C = [1 0];
D = 0;

sys = ss(A,B,C,D);
```

```
y_sim_ss = lsim(sys,u,t,[y(1) 24.49]);

%% Transform

H = tf(K*wn.^2, [1, 2*zeta*wn, wn.^2]);
y_sim_tf = lsim(H,u,t);

%% Filtering
idx_val = (tmax+1):length(t); % Unused values

% Moving average filter
windowSize = 55;
y_mov = movmean(y,windowSize);

% Butterworth filter
fs = 1/Ts;
fc = 5;
ord = 4;
[b_filt, a_filt] = butter(ord, fc/(fs/2), 'low');
y_butter = filtfilt(b_filt, a_filt, y);

%% Performance indexes
% J = ||eps||_2 / sqrt(N)
% eps_MPN = ||eps||_2 / ||y - mean(y)||_2

% State Space - nefiltrat
J_ss_full    = norm(y_sim_ss - y) / sqrt(length(y));
eMPN_ss_full = norm(y_sim_ss - y) / norm(y - mean(y));
J_ss_val     = norm(y_sim_ss(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_ss_val  = norm(y_sim_ss(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% State Space - movmean
J_ss_mov_full    = norm(y_sim_ss - y_mov) / sqrt(length(y_mov));
eMPN_ss_mov_full = norm(y_sim_ss - y_mov) / norm(y_mov - mean(y_mov));
J_ss_mov_val     = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_ss_mov_val  = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% State Space - Butterworth
J_ss_butter_full    = norm(y_sim_ss - y_butter) / sqrt(length(y_butter));
eMPN_ss_butter_full = norm(y_sim_ss - y_butter) / norm(y_butter - mean(y_butter));
J_ss_butter_val     = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_ss_butter_val  = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

% Transfer Function - nefiltrat
J_tf_full    = norm(y_sim_tf - y) / sqrt(length(y));
eMPN_tf_full = norm(y_sim_tf - y) / norm(y - mean(y));
J_tf_val     = norm(y_sim_tf(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_tf_val  = norm(y_sim_tf(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% Transfer Function - movmean
J_tf_mov_full    = norm(y_sim_tf - y_mov) / sqrt(length(y_mov));
eMPN_tf_mov_full = norm(y_sim_tf - y_mov) / norm(y_mov - mean(y_mov));
J_tf_mov_val     = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_tf_mov_val  = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% Transfer Function - Butterworth
J_tf_butter_full    = norm(y_sim_tf - y_butter) / sqrt(length(y_butter));
eMPN_tf_butter_full = norm(y_sim_tf - y_butter) / norm(y_butter - mean(y_butter));
J_tf_butter_val     = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_tf_butter_val  = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

%% Plots - State Space model
figure
subplot(3,1,1)
plot(t,y_sim_ss,'r', t, y, 'g');
title('State Space model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_full, 100*eMPN_ss_full))

subplot(3,1,2)
plot(t,y_sim_ss,'r', t, y_mov, 'g');
```

```
title('State Space model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_mov_full, 100*eMPN_ss_mov_full))

subplot(3,1,3)
plot(t,y_sim_ss,'r', t, y_butter, 'g');
title('State Space model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_butter_full, 100*eMPN_ss_butter_full))

%% Plots - Transfer Function model
figure
subplot(3,1,1)
plot(t,y_sim_tf,'r', t, y, 'g');
title('Transfer Function model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_full, 100*eMPN_tf_full))

subplot(3,1,2)
plot(t,y_sim_tf,'r', t, y_mov, 'g');
title('Transfer Function model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_mov_full, 100*eMPN_tf_mov_full))

subplot(3,1,3)
plot(t,y_sim_tf,'r', t, y_butter, 'g');
title('Transfer Function model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_butter_full, 100*eMPN_tf_butter_full))
```